Python Engineering

WEEK 02 - Data Structures | CLASS 08 - Data Processing | EX16

Cambio oficial de clase: class_07_numpy_arrays queda completada. Seguimos en week_02 y entramos en **class_08_data_processing**.

EX16 - Procesado de trayectoria con mascaras y estadistica

Ahora que sabemos construir y manipular arrays, el siguiente paso es usarlos para responder preguntas sobre datos: que muestras corresponden al ascenso, cuales estan por encima de cierta altura, que ocurre en una ventana temporal y que estadisticas resumen la trayectoria.

1. Partimos de trajectory_array

```
# Convention kept from EX15
# columns: [h, v, t]

heights = trajectory_array[:, 0]
velocities = trajectory_array[:, 1]
times = trajectory_array[:, 2]
```

2. Boolean masks

```
ascending_mask = velocities > 0
descending_mask = velocities < 0
above_100_mask = heights > 100
```

Una mascara booleana es un array de True/False con la misma longitud que los datos. NumPy puede usarla para seleccionar solo las filas donde la condicion es True.

```
ascending_states = trajectory_array[ascending_mask]
above_100_states = trajectory_array[above_100_mask]
```

3. Combinar condiciones

```
high_and_ascending = (
    (heights > 50)
    & (velocities > 0)
)

selected = trajectory_array[high_and_ascending]
```

Con arrays NumPy se usan `&`, `|` y `~`, no los operadores escalares and/or/not.

4. Seleccion por ventana temporal

```
time_window = (times >= 2.0) & (times <= 5.0)
window_states = trajectory_array[time_window]
```

5. Estadisticas

```
mean_height = np.mean(heights)
std_height = np.std(heights)
min_velocity = np.min(velocities)
max_velocity = np.max(velocities)
```

Las estadisticas describen conjuntos completos de muestras. Recuerda que su significado fisico depende de que las muestras representen correctamente el fenomeno.

6. np.where()

```
indices_above_100 = np.where(heights > 100)[0]

print(indices_above_100)
```

`np.where(condition)` permite recuperar las posiciones donde una condicion se cumple.

7. Crear una columna derivada

```
energy = (
    config['g'] * heights
    + 0.5 * velocities**2
)

processed_data = np.column_stack((
    heights,
    velocities,
    times,
    energy
))
```

Ahora processed_data tiene 4 columnas: `[h, v, t, energy]`. Esto es un patron habitual: partir de datos originales y generar variables derivadas.

8. Diferencias entre muestras

```
dt_samples = np.diff(times)
dh_samples = np.diff(heights)
```

`np.diff()` calcula diferencias consecutivas: x[i+1] - x[i]. El resultado tiene un elemento menos que el array original.

9. Tu mision

```
Create: ex16_trajectory_data_processing.py

1. Reuse the EX15 trajectory_array.
2. Extract h, v, t columns.
3. Create ascent and descent masks.
4. Count ascent/descent states with np.sum(mask).
5. Select states with h > 100 m.
6. Select states between t = 2 s and t = 5 s.
7. Create a combined mask: h > 50 AND v > 0.
8. Compute mean/std of height and mean/std of velocity.
9. Use np.where() to get indices with h > 100 m.
10. Recompute specific energy and add it as a fourth column.
11. Use np.diff(times) and verify dt is approximately constant.
12. Print a compact engineering summary.
```

10. Preguntas que debes poder responder

```
- How many samples belong to ascent?
- How many belong to descent?
- How many samples are above 100 m?
- Which indices are above 100 m?
- What is the mean and standard deviation of altitude?
- Is dt constant according to np.diff(times)?
- What is processed_data.shape after adding energy?
```

11. Criterio de cierre

EX16 estara completado cuando puedas convertir una pregunta de ingenieria en una condicion booleana, usar esa condicion para filtrar el array y obtener una respuesta numerica sin escribir loops innecesarios.